



Assessment

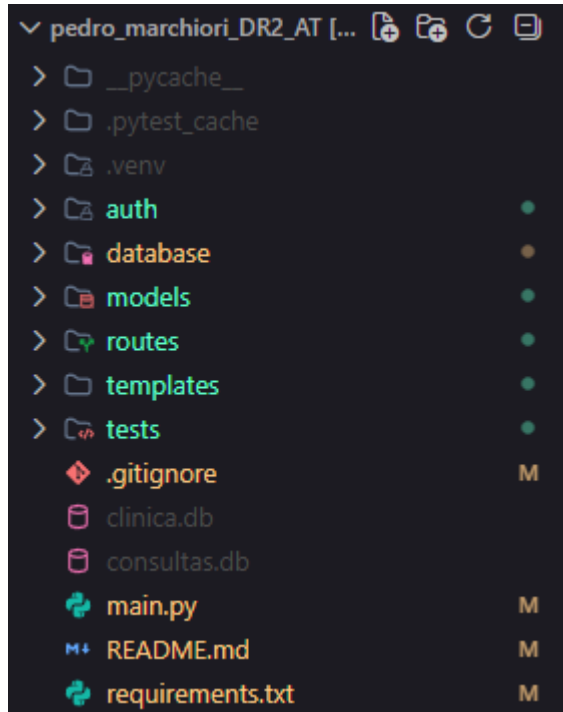
Disciplina: Desenvolvimento Seguro de Aplicações Web

Docente: Fabiano Domingues
Discente: Pedro Marchiori de Araújo
Data entrega: 03/10/2025

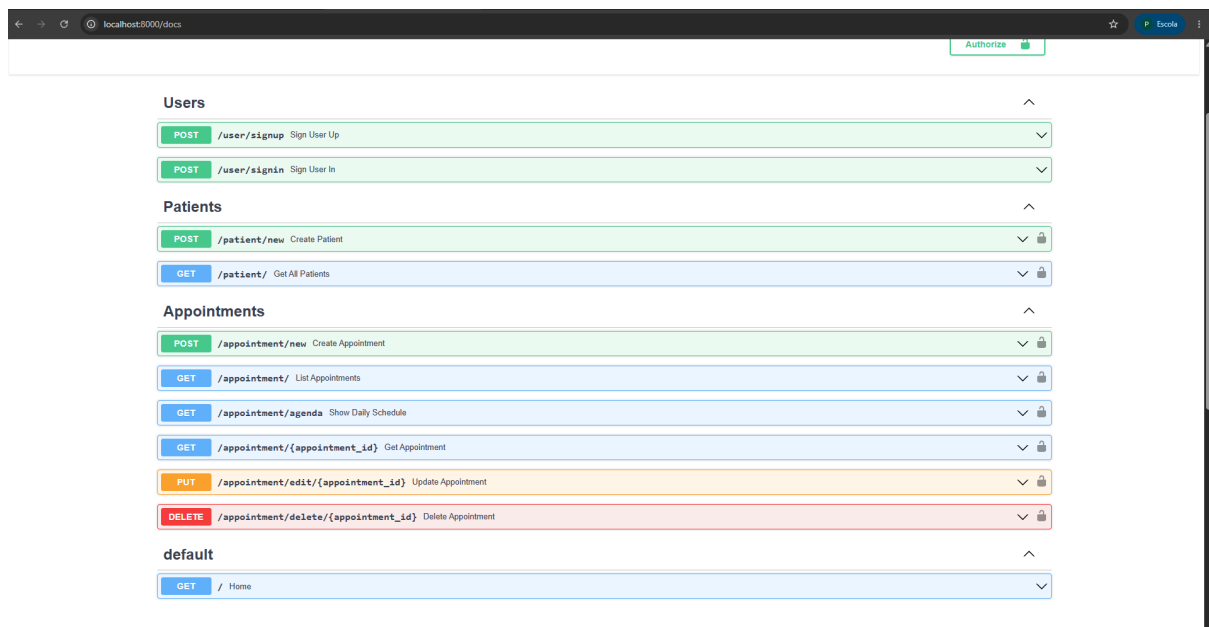
Vídeo de apresentação: <https://youtu.be/Hh0ox9UgJaw>

Exercício 1 - Fundação da API de agendamento

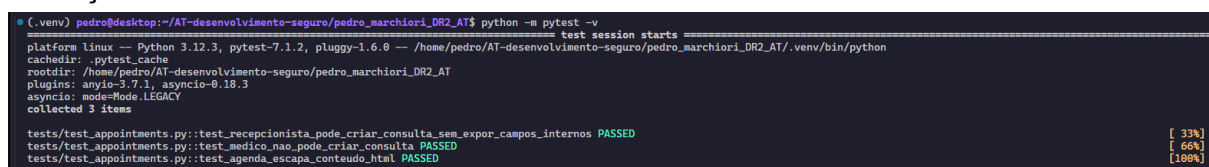
Estrutura inicial do projeto:



CRUDS iniciais:



Execução do testes:

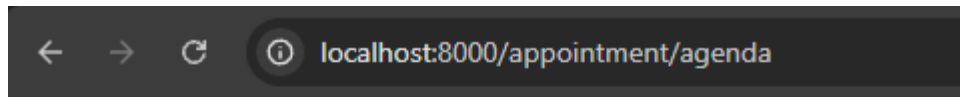


Exercício 2 - Controle de exposição de dados e templates seguros

Foi criado um response model específico para consultas, permitindo que a API retorne somente id, paciente, profissional, data e status. Campos internos ou sensíveis, como observações e data de criação, permanecem no banco, mas são excluídos das respostas JSON, reduzindo o risco de exposição acidental.

Tela de tela html simples implementada com Jinja2:

Teste de XSS.



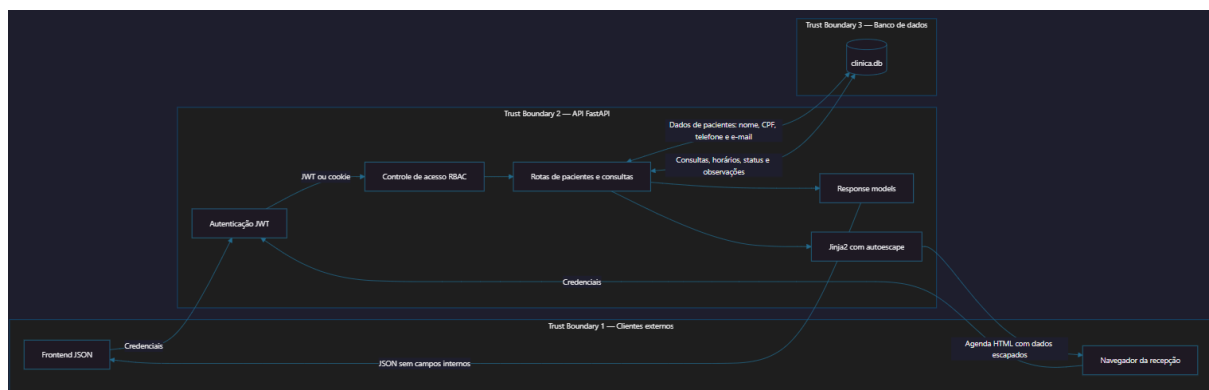
Agenda de 01/10/2026

Horário	Paciente	Profissional	Status
03:14	0	0	<script>alert('xss')</script>

Exercício 3 - Fundamentos de segurança e modelagem inicial

Análise da tríade CIA, DFD e tabela de Trust Boundaries e fluxos sensíveis disponível em: /docs/ex3_analise_cia_dfd.md

DFD:



Exercício 4 - Modelagem de ameaças com STRIDE

Disponível em: /docs/ex4_threat_model_stride.md

Exercício 5 - Arquitetura de segurança e vetores de ataque

Disponível em: /docs/ex5_arquitetura_seguranca.md

Exercício 6 - Autenticação e autorização

Foi adotado OAuth2PasswordBearer com JWT de duração limitada e hashing bcrypt para senhas. O sistema nunca precisa comparar ou armazenar senhas em texto puro. Para administradores, foi implementado um segundo fator simulado: o login inicial gera uma autorização temporária, e o token de acesso administrativo só é emitido após a validação do código MFA.

O modelo de autorização combina RBAC com autorização por recurso. O RBAC diferencia recepcionistas, profissionais e administradores, enquanto o ownership garante que profissionais criem e gerenciem apenas consultas de pacientes vinculados a eles. Essa combinação é mais adequada que RBAC isolado, pois dois usuários com o mesmo papel não devem necessariamente acessar os mesmos registros.

Exercício 7 - Escopos e integração externa

Para a comunicação máquina-a-máquina foi escolhido o fluxo OAuth 2.0 Client Credentials. O laboratório autentica-se com credenciais próprias e recebe um JWT de curta duração contendo `actor_type="laboratory"` e um escopo restrito à consulta de disponibilidade.

O escopo `appointments:availability:read` permite somente consultar horários disponíveis, sem expor pacientes, profissionais, observações ou consultas completas. Tokens humanos e tokens do laboratório possuem claims diferentes e são rejeitados quando utilizados no tipo errado de rota, reduzindo o impacto caso uma credencial do parceiro seja comprometida.

Exercício 8 - Identificação de vulnerabilidades OWASP Top 10

Disponível em: `/docs/ex8_vulnerabilidades_owasp.md`

Exercício 9 - Correção de vulnerabilidades de entrada e saída

No Exercício 9, as principais decisões de segurança foram centralizar a validação dos JWTs em um middleware e reutilizar uma única verificação de ownership nas rotas de consultas, reduzindo o risco de BOLA. Os modelos de entrada passaram a usar Pydantic com `extra="forbid"` (por meio do `SQLModel`), `whitelist`, `regex` e limites de tamanho. As consultas ao banco permanecem parametrizadas pelo `SQLModel`, enquanto o `autoescape` do Jinja2 protege a página HTML contra XSS armazenado.

O endpoint adicional identificado foi `POST /user/signup`, que compartilhava o comportamento de aceitar e ignorar silenciosamente campos não declarados. Após a correção, seu modelo de entrada também herda de `StrictInputModel`, fazendo com que tentativas de enviar propriedades indevidas, como `role: "admin"`, sejam rejeitadas com 422 `Unprocessable Entity`.

Exercício 10 - Hardening de rede e proteção contra abuso

CORS foi configurado com uma lista explícita de origens autorizadas.

Também foi criado um middleware para adicionar os cabeçalhos HSTS, X-Frame-Options e X-Content-Type-Options. Esses cabeçalhos impedem que a aplicação seja carregada dentro de frames e evitam que o navegador tente interpretar respostas com um tipo de conteúdo diferente do declarado.

Por fim, foi aplicado um rate limiting específico ao endpoint de login. Após cinco credenciais inválidas dentro de 60 segundos para o mesmo endereço de origem, novas tentativas recebem 429 Too Many Requests e o cabeçalho Retry-After. Essa limitação reduz o risco de ataques de força bruta sem impor o mesmo limite às demais rotas da aplicação.

Verificação dos cabeçalhos de segurança:

```
pedro@desktop:~$ curl -I http://127.0.0.1:8000/
HTTP/1.1 405 Method Not Allowed
date: Fri, 02 Oct 2026 21:10:50 GMT
server: uvicorn
allow: GET
content-length: 31
content-type: application/json
strict-transport-security: max-age=31536000; includeSubDomains
x-frame-options: DENY
x-content-type-options: nosniff
```

Teste do CORS com origem autorizada:

```
pedro@desktop:~$ curl -i -X OPTIONS http://127.0.0.1:8000/user/signin \
-H "Origin: http://localhost:3000" \
-H "Access-Control-Request-Method: POST"
HTTP/1.1 200 OK
date: Fri, 02 Oct 2026 21:11:49 GMT
server: uvicorn
vary: Origin
access-control-allow-methods: GET, POST, PUT, DELETE, OPTIONS
access-control-max-age: 600
access-control-allow-headers: Accept, Accept-Language, Authorization, Content-Language, Content-Type
access-control-allow-credentials: true
access-control-allow-origin: http://localhost:3000
content-length: 2
content-type: text/plain; charset=utf-8
strict-transport-security: max-age=31536000; includeSubDomains
x-frame-options: DENY
x-content-type-options: nosniff
```

Teste do CORS com origem não autorizada:

```
OKpedro@desktop:~$ curl -i -X OPTIONS http://127.0.0.1:8000/user/signin \
-H "Origin: https://origem-invalida.exemplo" \
-H "Access-Control-Request-Method: POST"
HTTP/1.1 400 Bad Request
date: Fri, 02 Oct 2026 21:13:14 GMT
server: uvicorn
vary: Origin
access-control-allow-methods: GET, POST, PUT, DELETE, OPTIONS
access-control-max-age: 600
access-control-allow-headers: Accept, Accept-Language, Authorization, Content-Language, Content-Type
access-control-allow-credentials: true
content-length: 22
content-type: text/plain; charset=utf-8
strict-transport-security: max-age=31536000; includeSubDomains
x-frame-options: DENY
x-content-type-options: nosniff

Disallowed CORS originpedro@desktop:~$ |
```

Exercício 11 - Persistência segura

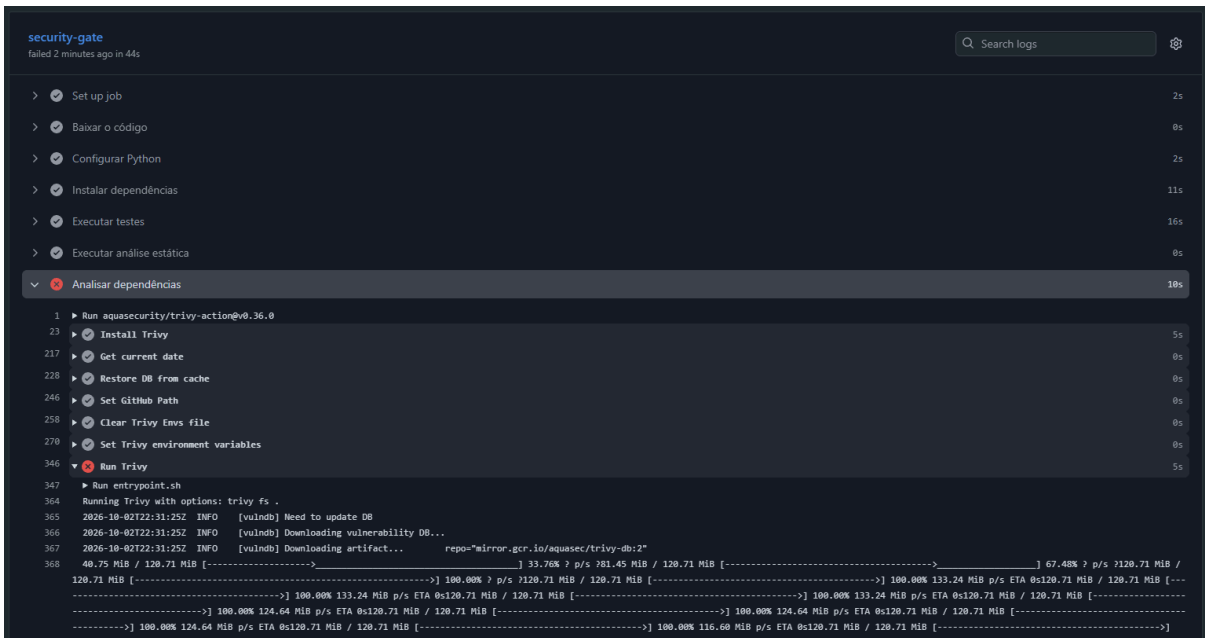
A aplicação desenvolvida até o momento já utilizava SQLAlchemy com um banco SQLite persistente, consultas parametrizadas e sessões fornecidas às rotas por injeção de dependência. Dessa forma, valores recebidos dos usuários são enviados separadamente ao banco, sem concatenação de strings SQL, reduzindo o risco de SQL injection.

A configuração da conexão foi transferida para uma classe baseada em BaseSettings, que lê DATABASE_URL do ambiente ou do arquivo .env. Assim, eventuais credenciais de bancos externos não precisam permanecer no código-fonte, enquanto o .env.example documenta a configuração necessária.

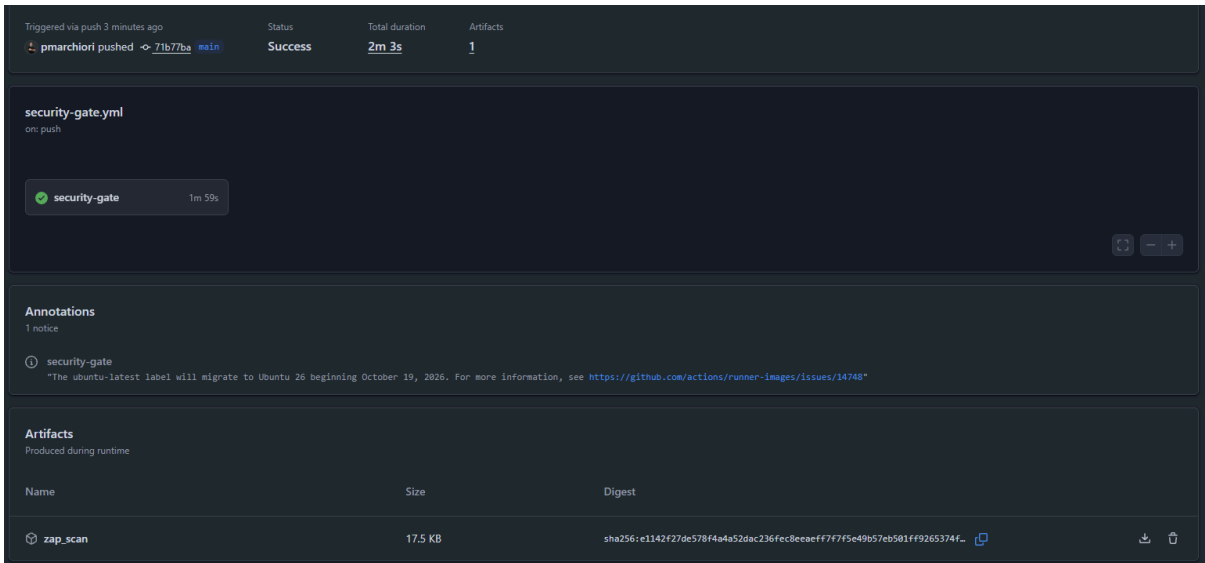
Exercício 12 - Pipeline DevSecOps e auditoria automatizada

Documentação disponível em `/docs/ex12_pipeline_devops.md`

Erro ao rodar pipeline pois o Trivy encontrou 5 vulnerabilidades altas e 2 críticas nas versões herdadas do starter kit:



Sucesso ao rodar após as correções necessárias:



Exercício 13 - Capstone: auditoria final e relatório de rastreabilidade
Relatório disponível em: [/docs/ex13_relatorio_final.md](#)